

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: Sec 1. Introduction To Python

Lecture: 7. Programming Paradigms

Section 1: Lecture Summary

This lecture introduces the concept of **programming paradigms**, which are methodologies or approaches used in software development to organize and write code effectively. It explains that Python supports multiple paradigms: **procedural programming, object-oriented programming, modular programming, and functional programming**. The instructor provides an overview of each paradigm, emphasizing their roles in managing complexity in larger programs. Procedural programming involves breaking a program into smaller functions, making debugging and development easier. Modular programming extends this by grouping functions into interchangeable modules. Object-oriented programming bundles data and functions into classes and objects, allowing a more natural and organized design. Finally, functional programming is briefly described as supporting mathematical functions, which is beneficial for data science and AI tasks with Python.

Section 2: Key Concepts and Explanations

- Programming Paradigms

Programming paradigms are comprehensive strategies or methodologies for structuring software applications. Different languages support different paradigms. Python supports multiple paradigms, making it flexible.

- Procedural Programming

- Involves writing a sequence of instructions or code lines to perform tasks.
- Monolithic programs: large single-block programs where debugging is difficult.
- Solution: break the program into smaller pieces called **functions or procedures**, each performing a minor or complete task.

- Benefits: Easier debugging, focused development on parts, supports team collaboration by dividing functions among developers.

- Function = group of instructions performing a specific task.

- Example visualization: main program calls many smaller functions.

- **Modular Programming**

- Extension of procedural programming.

- Groups related functions into **modules or libraries**.

- Key advantage: modules can be replaced or upgraded independently without affecting the entire program.

- Analogous to replacing components in engineering devices.

- Modules contain functions, procedures (also called routines), objects, and variables.

- **Object-Oriented Programming (OOP)**

- Focuses on combining data and the operations on that data together.

- Introduces **classes** as blueprints bundling variables (data) and functions (methods) together.

- Objects are instances of classes, containing both data and behavior.

- Benefits: Encapsulation, easier management of related data and functions, reusable blueprints.

- Example: A class `Account` with data members (variables) and methods (functions operating on those variables).

- Python is naturally object-oriented – everything is an object and built-in classes are extensively used.

- **Functional Programming**

- Inspired by mathematical functions where output depends solely on input ($f(x)$).

- Promotes writing functions with no side effects or mutable state.

- Python supports functional programming concepts and provides many built-in mathematical/statistical functions.

- Important for data science, machine learning, and AI applications.

- Relationship Between Paradigms in Python

- Python strongly supports **procedural programming**, making it easy to write functions and work step-by-step.

- It fully supports **object-oriented programming**, where everything can be treated as objects of classes.

- It supports **modular programming**, encouraging breaking down large programs into independent modules.

- It supports **functional programming** principles that aid in mathematical and statistical computations.

Section 3: Example Code and Use Cases

Example sketch of procedural programming: using functions to break the program

perform small task 1

perform small task 2

main program calls various functions

```
def task1():
    print("Task 1 completed")

def task2():
    print("Task 2 completed")

def main_program():
    task1()
    task2()

main_program()
```

Explanation: This example shows how a large program can be broken down into smaller functions called separately to organize code clearly.

Rough sketch of an Object-Oriented Programming example - Class and Object

Creating object of class Account

```
class Account:
    def __init__(self, owner, balance=0):
        self.owner = owner # data variable
        self.balance = balance # data variable

    def deposit(self, amount):
        self.balance += amount # operation on data
        print(f"{amount} deposited. New balance: {self.balance}")

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            print(f"{amount} withdrawn. New balance: {self.balance}")
        else:
            print("Insufficient funds")

acc = Account("John Doe", 1000)
acc.deposit(500)
acc.withdraw(200)
```

Explanation: This shows how data (owner, balance) and operations (deposit, withdraw) are bundled in a class, creating an object to use those functions and variables.

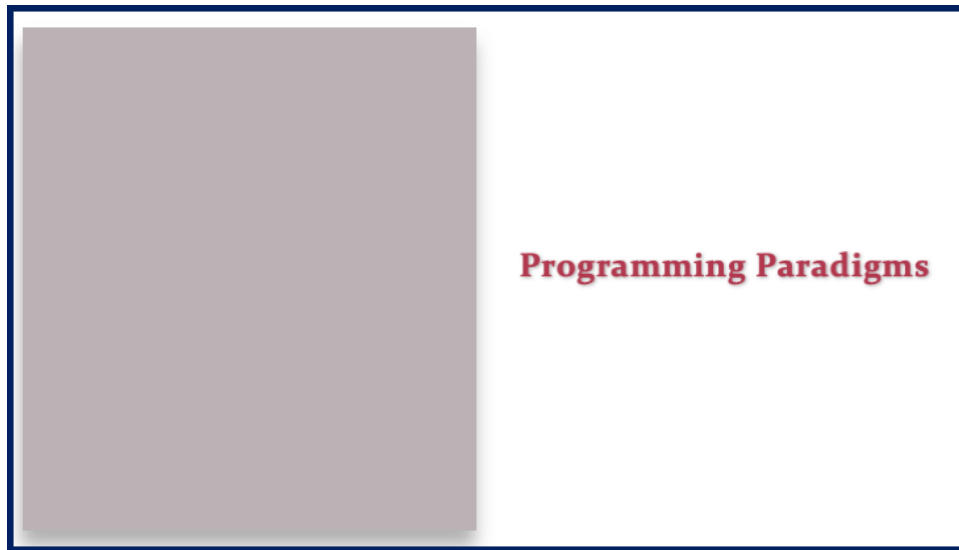
Section 4: Key Takeaways

- Programming paradigms are essential methodologies for structuring and developing software efficiently.
- Python supports four major paradigms: procedural, modular, object-oriented, and functional.
- Procedural programming involves breaking a large program into smaller functions for manageability.

- Modular programming groups functions into independent modules that can be updated or replaced easily.
- Object-oriented programming combines data and functions in classes and uses objects for real-world modeling and reuse.
- Functional programming in Python leverages mathematical functions — important for statistics and AI-related fields.
- Understanding these paradigms helps manage complexity and collaborate effectively when developing large Python applications.

Lecture in Slides

Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.



Programming Paradigms

Programming Paradigms

- Procedural Programming



Programming Paradigms

- Procedural Programming
- Object-Oriented Programming



Programming Paradigms

- Procedural Programming
- Object-Oriented Programming
- Modular Programming



Programming Paradigms

- Procedural Programming
- Object-Oriented Programming
- Modular Programming
- Functional Programming



Programming Paradigms

Procedural Programming



Programming Paradigms
Procedural Programming

Program



Programming Paradigms
Procedural Programming

Program

Programming Paradigms

Procedural Programming

Program

Programming Paradigms

Procedural Programming

Program



Programming Paradigms
Procedural Programming

Program

The image shows a presentation slide with a dark blue border. On the left side, there is a large, solid grey rectangle. On the right side, the title 'Programming Paradigms' is written in a large, bold, red serif font. Below it, the subtitle 'Procedural Programming' is written in a smaller, regular, grey sans-serif font. Further down, the word 'Program' is centered in a bold, black serif font. Below 'Program', there are ten horizontal black lines, serving as a template for code or text.

Programming Paradigms

Procedural Programming

Program

[illegible]

Programming Paradigms

Procedural Programming

Program

[illegible][illegible][illegible]

Procedural Programming

[illegible]

Procedural Programming

[illegible]

Programming Paradigms

Procedural Programming

Program

```
def func(<parameter>):  
    _____  
    _____  
  
    return result
```

[illegible]

```
def fun1(<parameter>):
    _____
    _____
    _____
    return result
```

The diagram illustrates the relationship between Programming Paradigms and Program. It features a large gray box on the left, a central box containing a function definition, and a large white box on the right labeled "Program" with horizontal lines representing code.

Programming Paradigms
Procedural Programming

Program

```
def func(parameter):  
    _____  
    _____  
    _____  
    return result
```

[illegible]

```
def fun1(<parameter>):
    _____
    _____
    _____
    return result
```


[illegible]

[illegible]

The diagram illustrates the relationship between a Program and a Function in Procedural Programming. It is divided into two main sections: a large gray area on the left and a white area on the right.

Left Section (Gray Area): This area is mostly empty, representing the main body of a program.

Right Section (White Area): This section contains the following elements:

- Header:** "Programming Paradigms" in red, with "Procedural Programming" in blue below it.
- Title:** "Program" in bold black text.
- Code Snippets:**
 - A function definition: `def func(parameter):` followed by three lines of code, ending with `return result`.
 - A function call: `func()`.
- Flow Arrows:** A series of horizontal arrows pointing from the function definition box to the function call, indicating the flow of execution.

[illegible]

```
def fun1(<parameter>):  
    _____  
    _____  
    _____  
    return result
```

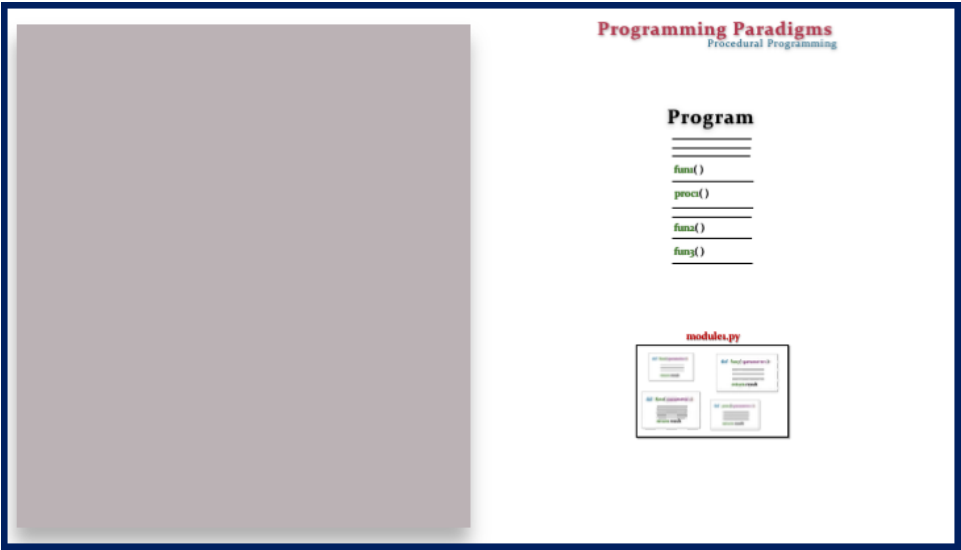
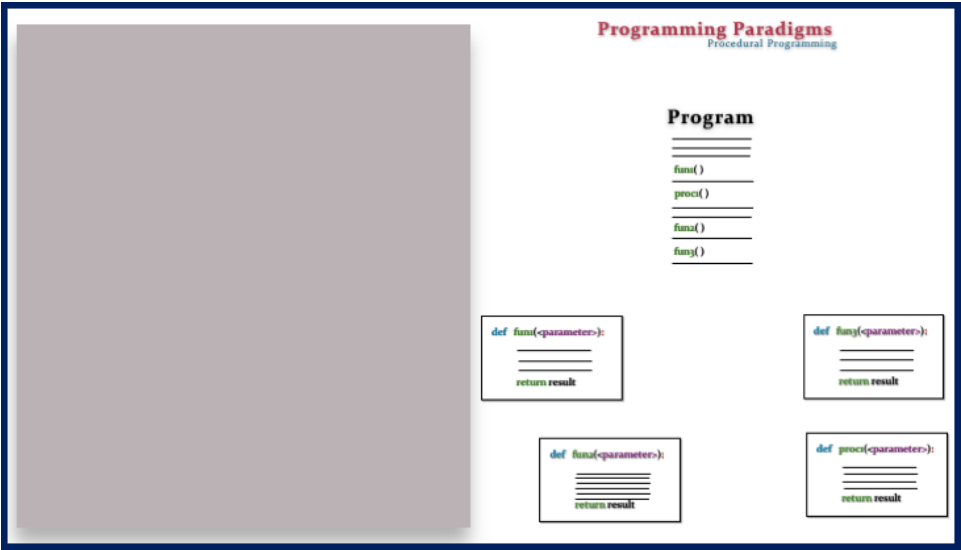
Programming Paradigms

Program

fun()proci()

```
def fun1(<parameter>):  
    _____  
    _____  
    _____  
    return result
```

```
def proci(<parameter>):
    _____
    _____
    _____
    return result
```

[illegible]

```
fun1()
proc1()
fun2()
fun3()
```

Four small diagrams illustrating different ways to represent the same data structure. Each diagram has a title and a list of items.

- Diagram 1:** Title: List. Items: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10.
- Diagram 2:** Title: List. Items: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10.
- Diagram 3:** Title: List. Items: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10.
- Diagram 4:** Title: List. Items: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10.

Program

fun1()
proc1()
fun2()
fun3()

module1.py

module1
module2
module3
module4

module2.py

module5
module6

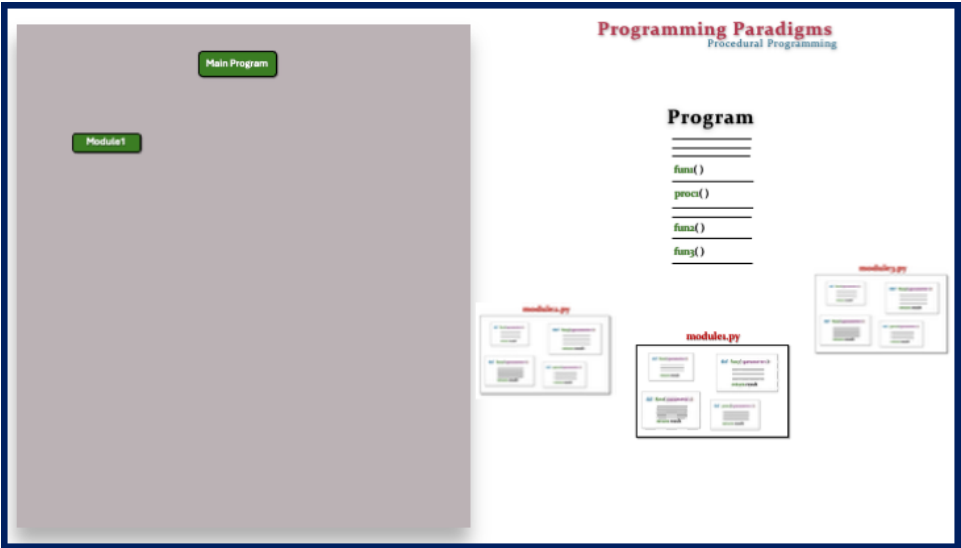
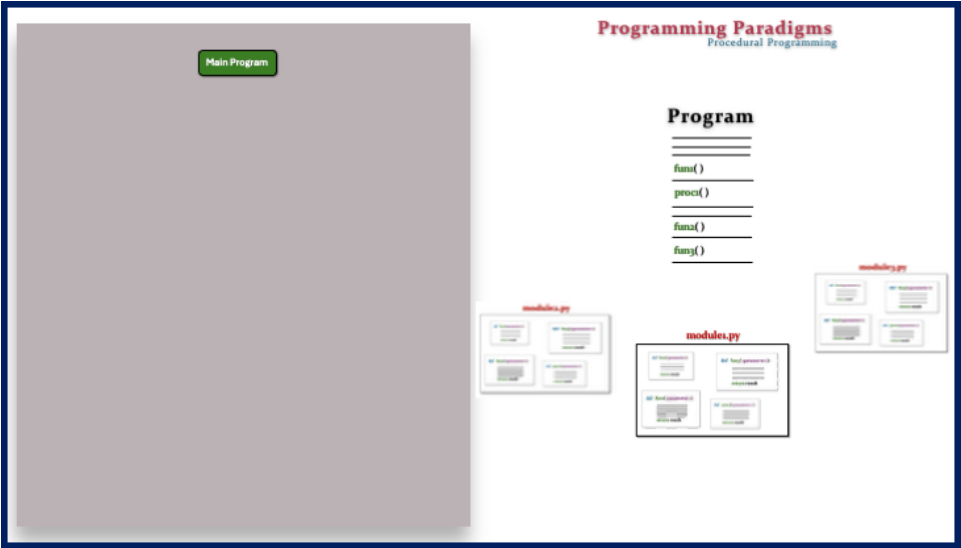
module3.py

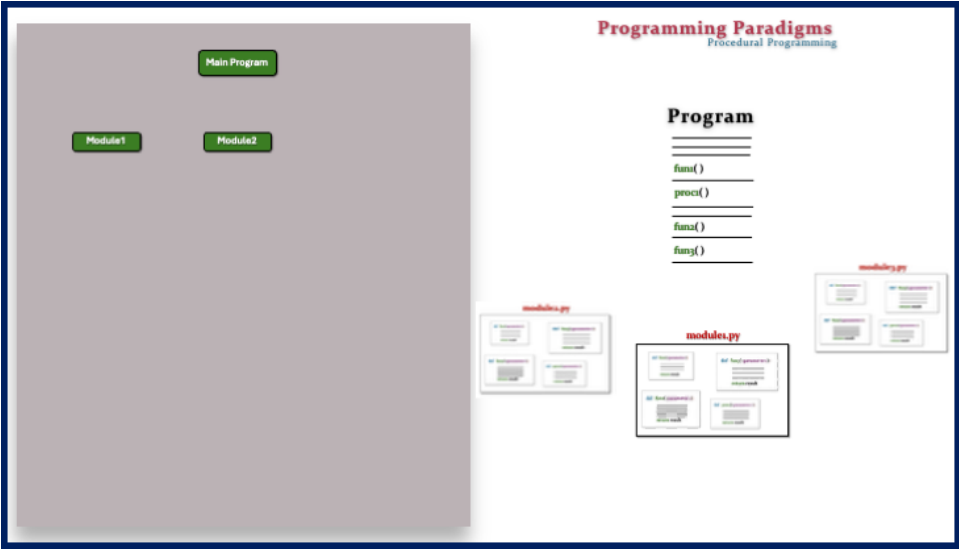
module7
module8

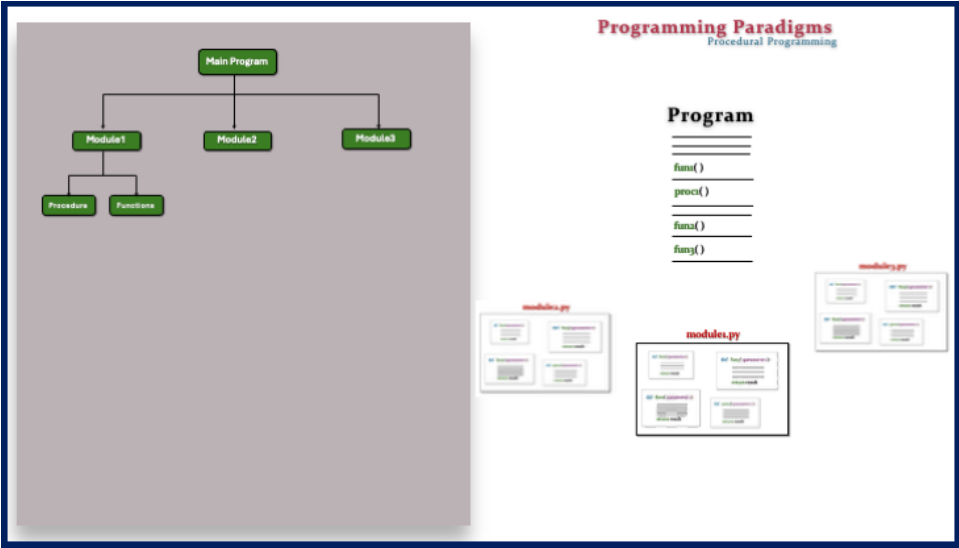
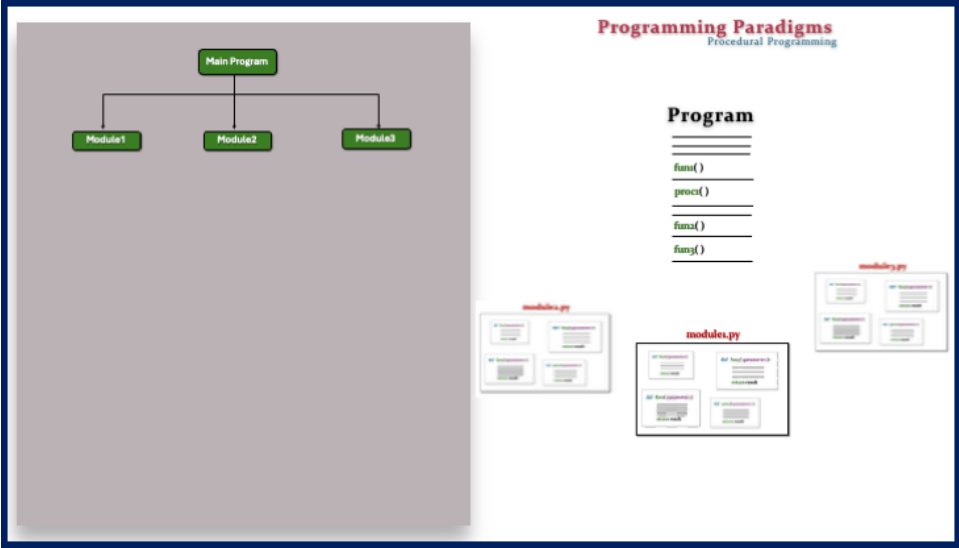
```
fun1()
proc1()
fun2()
fun3()
```

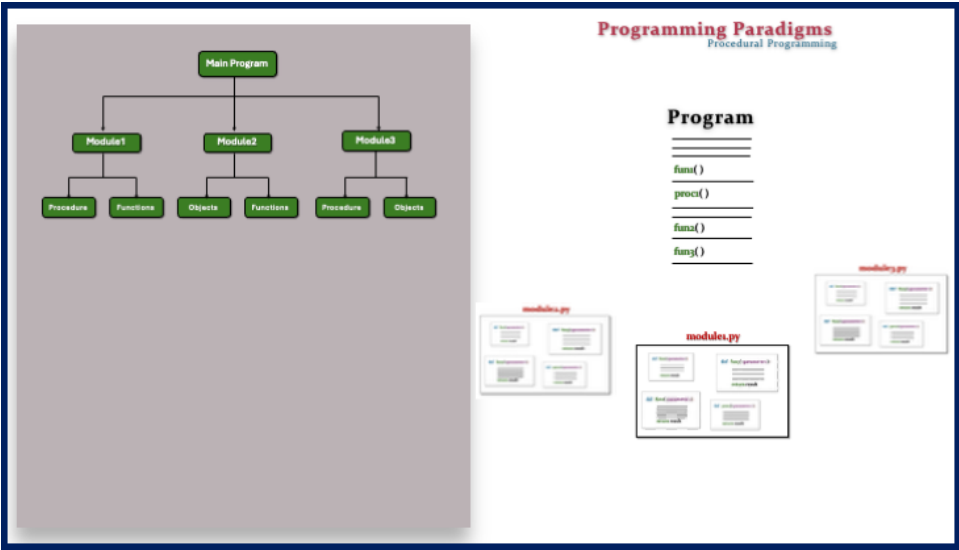
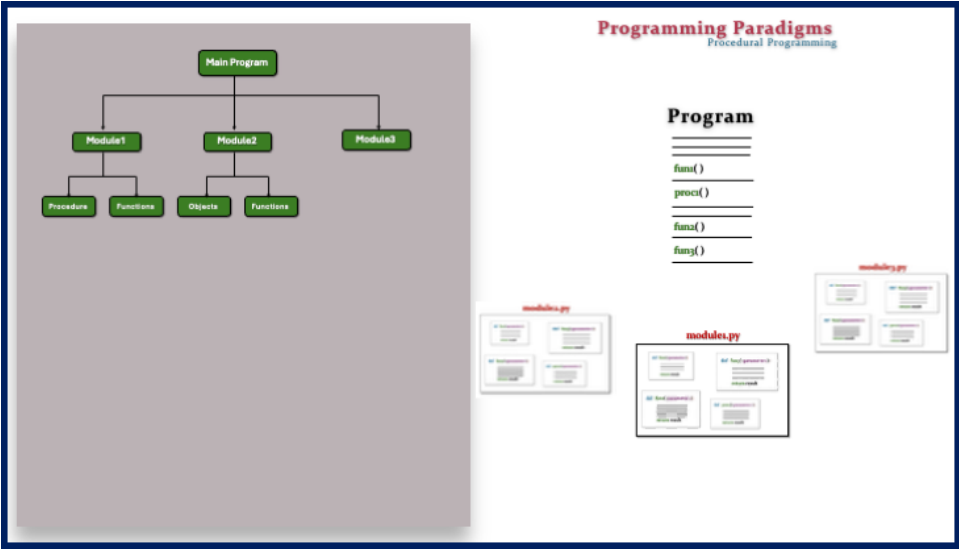
Four cards showing the results of the four basic arithmetic operations:

- Addition:** $100 + 100 = 200$
- Subtraction:** $100 - 100 = 0$
- Multiplication:** $100 \times 100 = 10000$
- Division:** $100 \div 100 = 1$











Programming Paradigms

- Object-Oriented Programming



Programming Paradigms
Object-Oriented Programming

Programming Paradigms

Object-Oriented Programming

class Account

Programming Paradigms

Object-Oriented Programming

class Account

Programming Paradigms

Object-Oriented Programming

class Account

data variables

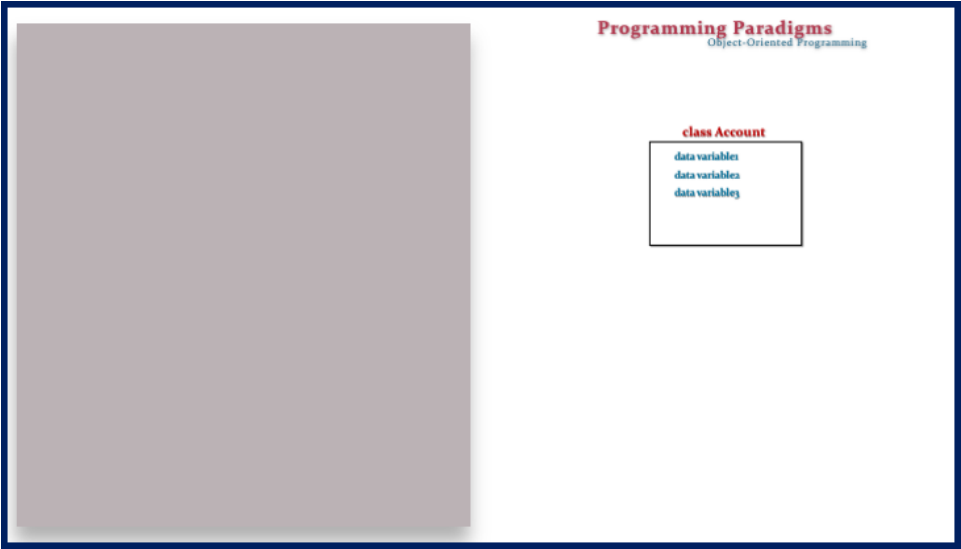
Programming Paradigms

Object-Oriented Programming

class Account

data variables

data variables



Programming Paradigms

Object-Oriented Programming

class Account

data variables
data variable
data variable

```
def __init__(self):  
    _____  
    return result
```

Programming Paradigms

Object-Oriented Programming

class Account

data variables
data variable
data variable

```
def __init__(self):  
    _____  
    return result
```

```
def __init__(self):  
    _____  
    return result
```

Programming Paradigms

Object-Oriented Programming

class Account

data variables
data variables
data variables

```
def handleWithdraw():  
    =====  
    return result
```

```
def handleDeposit():  
    =====  
    return result
```

```
def handleTransfer():  
    =====  
    return result
```

Programming Paradigms

Object-Oriented Programming

class Account

data variables
data variables
data variables

```
def handleWithdraw():  
    =====  
    return result
```

```
def handleDeposit():  
    =====  
    return result
```

```
def handleTransfer():  
    =====  
    return result
```

```
def handleWithdraw():  
    =====  
    return result
```

```
def handleDeposit():  
    =====  
    return result
```


Programming Paradigms

Object-Oriented Programming

class Account

data variables

data variablea

data variablej

def handlequerymethod():

=====

return result

def handlequerymethod():

=====

return result

def handlequerymethod():

=====

return result

def handlequerymethod():

=====

return result

Programming Paradigms

Object-Oriented Programming

class Account

data variables

data variablea

data variablej

def handlequerymethod():

=====

return result

def handlequerymethod():

=====

return result

def handlequerymethod():

=====

return result

def handlequerymethod():

=====

return result

Program

Programming Paradigms

Object-Oriented Programming

class Account

data variables

data variablea

data variabley

def funcparametrizb

=====

return result

def funcparametrizb

=====

return result

def funcparametrizb

=====

return result

def funcparametrizb

=====

return result

Program

=====

Programming Paradigms

Object-Oriented Programming

class Account

data variables

data variablea

data variabley

def funcparametrizb

=====

return result

def funcparametrizb

=====

return result

def funcparametrizb

=====

return result

def funcparametrizb

=====

return result

Program

=====

acc = Account()

Programming Paradigms

Object-Oriented Programming

class Account

data variables
data variablea
data variabley

def funcparametric():
=====
return result

def funcparametric():
=====
return result

def funcparametric():
=====
return result

def funcparametric():
=====
return result

def funcparametric():
=====
return result

Program

=====
acc = Account()
=====
acc.fun1()

acc.fun1()

Programming Paradigms

Object-Oriented Programming

class Account

data variables
data variablea
data variabley

def funcparametric():
=====
return result

def funcparametric():
=====
return result

def funcparametric():
=====
return result

def funcparametric():
=====
return result

def funcparametric():
=====
return result

Program

=====
acc = Account()
=====
acc.fun1()
acc.fun1()

acc.fun1()

Programming Paradigms

Object-Oriented Programming

class Account

data variables
data variables
data variables

def func1(self):

return result

def func2(self):

return result

def func3(self):

return result

def func4(self):

return result

Program

acc = Account()

acc.fun1()

acc.fun2()

acc.fun3()

Programming Paradigms

Object-Oriented Programming

class Account

data variables
data variables
data variables

```
def __init__(self):  
    _____  
    return much
```

```
def __init__(self):  
    _____  
    return much
```

```
def __init__(self):  
    _____  
    return much
```

```
def __init__(self):  
    _____  
    return much
```

Program

```
acc = Account()
```

```
acc.fun1()
```

```
acc.fun2()
```

```
acc.fun3()
```

```
acc.fun4()
```

Programming Paradigms

- Functional Programming

Programming Paradigms

- Functional Programming

Mathematical Functions